

Module6_Lab1

October 9, 2026

```
[1]: import sqlite3
import pandas as pd

df_customers = pd.read_csv("store_customers.csv")
df_orders = pd.read_csv("store_orders.csv")

conn = sqlite3.connect("store.db")

df_customers.to_sql("customers", conn, if_exists="replace", index=False)
df_orders.to_sql("orders", conn, if_exists="replace", index=False)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 9
      5 df_orders = pd.read_csv("store_orders.csv")
      6
      7 conn = sqlite3.connect("store.db")
      8
----> 9 df_customers.to_sql("customers", conn, if_exists="replace", index=False)
     10 df_orders.to_sql("orders", conn, if_exists="replace", index=False)

NameError: name 'conn' is not defined
```

```
[2]: import sqlite3
import pandas as pd

df_customers = pd.read_csv("store_customers.csv")
df_orders = pd.read_csv("store_orders.csv")

conn = sqlite3.connect("store.db")

df_customers.to_sql("customers", conn, if_exists="replace", index=False)
df_orders.to_sql("orders", conn, if_exists="replace", index=False)

conn.close()
```

```
[3]: import sqlite3
import pandas as pd

conn = sqlite3.connect('store.db')
query_result = pd.read_sql_query("SELECT * FROM orders LIMIT 5;", conn)
conn.close()

print(query_result.to_markdown(index=False))
```

```
Cell In[3], line 8
    print(query_result.to_markdown(index=False))
~
_IncompleteInputError: incomplete input
```

```
[4]: import sqlite3
import pandas as pd

conn = sqlite3.connect('store.db')
query_result = pd.read_sql_query("SELECT * FROM orders LIMIT 5;", conn)
conn.close()

print(query_result.to_markdown(index=False)))
```

```
Cell In[4], line 8
    print(query_result.to_markdown(index=False)))
~
SyntaxError: unmatched ')'
```

```
[5]: import sqlite3
import pandas as pd

conn = sqlite3.connect('store.db')
query_result = pd.read_sql_query("SELECT * FROM orders LIMIT 5;", conn)
conn.close()

print(query_result.to_markdown(index=False))
```

order_id	customer_id	product	category	amount	order_date
5000	4	paper	office	14.09	2026-05-25
5001	15	planner	office	51.86	2026-05-08

5002	34	shelf	furniture	116.25	2026-06-03
5003	2	granola	snacks	19.72	2026-05-19
5004	17	monitor	electronics	284.8	2026-05-10

```
[6]: conn = sqlite3.connect('store.db')
query = """
SELECT *
FROM orders
WHERE amount > 100
ORDER BY amount DESC;
"""
df_result = pd.read_sql_query(query, conn)
conn.close()

print(df_result.to_markdown(index=False))
```

order_id	customer_id	product	category	amount	order_date
5153	20	dock	electronics	499	2026-05-25
5140	30	headset	electronics	295.36	2026-05-26
5047	901	headset	electronics	293.79	2026-05-15
5150	903	monitor	electronics	293.35	2026-06-06
5014	6	monitor	electronics	287.83	2026-05-22
5113	4	headset	electronics	286.65	2026-05-15
5034	29	headset	electronics	286.36	2026-05-14
5149	8	headset	electronics	286.3	2026-05-05
5004	17	monitor	electronics	284.8	2026-05-10
5174	11	headset	electronics	277.24	2026-05-26
5188	904	headset	electronics	273.16	2026-05-22
5075	11	monitor	electronics	272.19	2026-05-23

5065	9	headset	electronics	270.06	2026-05-13
5008	30	headset	electronics	267.39	2026-06-08
5137	35	monitor	electronics	262.47	2026-06-13
5050	14	monitor	electronics	262.04	2026-06-12
5031	24	monitor	electronics	259.96	2026-05-31
5162	6	dock	electronics	255.8	2026-06-01
5087	3	headset	electronics	254.01	2026-05-08
5139	21	headset	electronics	250.85	2026-05-05
5142	9	dock	electronics	246.64	2026-05-06
5070	13	chair	furniture	245.57	2026-06-02
5163	26	chair	furniture	239.24	2026-06-07
5054	10	dock	electronics	238.81	2026-06-03
5127	8	shelf	furniture	236.23	2026-06-07
5151	36	shelf	furniture	225.57	2026-05-30
5058	23	chair	furniture	223.51	2026-06-06
5189	29	shelf	furniture	216.22	2026-05-03
5073	38	shelf	furniture	216.03	2026-06-07
5079	30	dock	electronics	209.59	2026-05-02
5168	29	shelf	furniture	208.18	2026-05-04
5191	13	headset	electronics	204.08	2026-06-08
5083	9	shelf	furniture	202.94	2026-05-12
5022	3	dock	electronics	201.9	2026-06-05
5195	14	chair	furniture	201.59	2026-05-10
5146	6	monitor	electronics	200.87	2026-05-17

5169	4	monitor	electronics	199.18	2026-06-11
5017	3	headset	electronics	198.11	2026-05-15
5028	29	shelf	furniture	197.97	2026-05-15
5154	22	chair	furniture	194.1	2026-05-02
5158	29	headset	electronics	187.54	2026-06-11
5096	7	headset	electronics	185.92	2026-06-07
5091	38	chair	furniture	184.65	2026-05-10
5180	7	desk	furniture	184.11	2026-05-15
5067	10	headset	electronics	183.57	2026-05-20
5024	31	chair	furniture	181.41	2026-05-20
5136	10	dock	electronics	179.49	2026-05-23
5049	15	shelf	furniture	175.94	2026-06-09
5144	21	headset	electronics	175.32	2026-05-02
5011	12	headset	electronics	174.59	2026-06-04
5094	15	shelf	furniture	174.3	2026-05-03
5069	5	monitor	electronics	174.28	2026-06-14
5036	30	dock	electronics	172.32	2026-06-02
5132	9	headset	electronics	169.07	2026-06-08
5185	37	headset	electronics	164.27	2026-05-22
5101	34	dock	electronics	163.78	2026-05-22
5147	38	chair	furniture	159.23	2026-05-21
5055	28	monitor	electronics	158.68	2026-05-22
5184	12	dock	electronics	153.53	2026-05-23
5078	35	dock	electronics	153.44	2026-05-01

5084	13	shelf	furniture	148.29	2026-06-03
5038	11	chair	furniture	143.61	2026-05-19
5082	22	dock	electronics	142.57	2026-05-05
5089	8	dock	electronics	142.16	2026-05-21
5190	19	shelf	furniture	141.35	2026-06-05
5193	26	desk	furniture	140.77	2026-05-17
5095	11	desk	furniture	139.54	2026-05-12
5119	18	shelf	furniture	137.65	2026-05-17
5035	36	shelf	furniture	136.6	2026-06-03
5061	19	desk	furniture	134.11	2026-05-06
5125	35	chair	furniture	133.93	2026-06-02
5106	10	monitor	electronics	133.8	2026-05-15
5030	23	chair	furniture	125.12	2026-06-12
5121	11	shelf	furniture	122.56	2026-05-18
5143	30	dock	electronics	118.99	2026-05-13
5002	34	shelf	furniture	116.25	2026-06-03
5056	13	chair	furniture	114.97	2026-05-30
5025	36	chair	furniture	113.37	2026-06-14
5197	23	monitor	electronics	110.87	2026-06-10
5107	38	shelf	furniture	110.31	2026-05-17
5046	13	shelf	furniture	108.47	2026-05-28
5009	14	chair	furniture	107.2	2026-05-24
5007	36	dock	electronics	103.77	2026-05-21
5023	9	desk	furniture	100.27	2026-06-05

```
[7]: df_customers = pd.read_csv('store_customers.csv')
df_orders = pd.read_csv('store_orders.csv')

conn = sqlite3.connect('store.db')
df_customers.to_sql('customers', conn, if_exists='replace', index=False)
df_orders.to_sql('orders', conn, if_exists='replace', index=False)

SELECT
    category,
    COUNT(order_id) AS order_count,
    AVG(amount) AS average_amount
FROM orders
GROUP BY category
ORDER BY average_amount DESC;

df_result = pd.read_sql_query(query, conn)
conn.close()

print(df_result.to_markdown(index=False))
```

Cell In[7], line 9

```
category,
~
```

IndentationError: unexpected indent

```
[8]: df_customers = pd.read_csv('store_customers.csv')
df_orders = pd.read_csv('store_orders.csv')

conn = sqlite3.connect('store.db')
df_customers.to_sql('customers', conn, if_exists='replace', index=False)
df_orders.to_sql('orders', conn, if_exists='replace', index=False)

SELECT
category,
COUNT(order_id) AS order_count,
AVG(amount) AS average_amount
FROM orders
GROUP BY category
ORDER BY average_amount DESC;

df_result = pd.read_sql_query(query, conn)
conn.close()

print(df_result.to_markdown(index=False))
```

Cell In[8], line 10

```
COUNT(order_id) AS order_count,  
      ^
```

SyntaxError: invalid syntax

```
[9]: df_customers = pd.read_csv('store_customers.csv')  
df_orders = pd.read_csv('store_orders.csv')  
  
conn = sqlite3.connect('store.db')  
df_customers.to_sql('customers', conn, if_exists='replace', index=False)  
df_orders.to_sql('orders', conn, if_exists='replace', index=False)  
  
query = """  
SELECT  
    category,  
    COUNT(order_id) AS order_count,  
    AVG(amount) AS average_amount  
FROM orders  
GROUP BY category  
ORDER BY average_amount DESC;  
"""  
df_result = pd.read_sql_query(query, conn)  
conn.close()  
  
print(df_result.to_markdown(index=False))
```

category	order_count	average_amount
electronics	60	190.57
furniture	46	146.252
office	51	30.1325
snacks	43	14.6198

The electronics category earns the most, averaging at \$190.57 per order.

```
[10]: df_customers = pd.read_csv('store_customers.csv')  
df_orders = pd.read_csv('store_orders.csv')  
  
conn = sqlite3.connect('store.db')  
df_customers.to_sql('customers', conn, if_exists='replace', index=False)  
df_orders.to_sql('orders', conn, if_exists='replace', index=False)  
  
query = """  
SELECT  
    c.city,  
    SUM(o.amount) AS total_spend
```

```

FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_spend DESC;
"""

```

```

df_result = pd.read_sql_query(query, conn)
conn.close()

print(df_result.to_markdown(index=False))

```

city	total_spend
Los Angeles	5862.99
Culver City	5316.88
Santa Monica	4017.93
Inglewood	2718.25
Torrance	1037.25
Venice	462.36

```

[11]: query = """
SELECT
    c.customer_id,
    c.name,
    c.city,
    COUNT(o.order_id) AS total_orders,
    ROUND(SUM(o.amount), 2) AS lifetime_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY lifetime_spend DESC
LIMIT 5;
"""

top_customers_df = pd.read_sql_query(query, conn)

conn.close()

print(top_customers_df)

```

```

-----
ProgrammingError                                Traceback (most recent call last)
Cell In[11], line 14
     10 GROUP BY c.customer_id, c.name, c.city
     11 ORDER BY lifetime_spend DESC
     12 LIMIT 5;
     13 """
--> 14 top_customers_df = pd.read_sql_query(query, conn)
     15

```

```

16 conn.close()
17

File ~\anaconda3\Lib\site-packages\pandas\io\sql.py:497, in read_sql_query(sql,
↳ con, index_col, coerce_float, params, parse_dates, chunksize, dtype,
↳ dtype_backend)
    494 assert dtype_backend is not lib.no_default
    496 with pandasSQL_builder(con) as pandas_sql:
--> 497     return pandas_sql.read_query(
    498         sql,
    499         index_col=index_col,
    500         params=params,
    501         coerce_float=coerce_float,
    502         parse_dates=parse_dates,
    503         chunksize=chunksize,
    504         dtype=dtype,
    505         dtype_backend=dtype_backend,
    506     )

File ~\anaconda3\Lib\site-packages\pandas\io\sql.py:2766, in SQLiteDatabase.
↳ read_query(self, sql, index_col, coerce_float, parse_dates, params, chunksize,
↳ dtype, dtype_backend)
    2755 def read_query(
    2756     self,
    2757     sql,
    (...) 2764     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
    2765 ) -> DataFrame | Iterator[DataFrame]:
-> 2766     cursor = self.execute(sql, params)
    2767     columns = [col_desc[0] for col_desc in cursor.description]
    2769     if chunksize is not None:

File ~\anaconda3\Lib\site-packages\pandas\io\sql.py:2700, in SQLiteDatabase.
↳ execute(self, sql, params)
    2698     raise TypeError("Query must be a string unless using sqlalchemy.")
    2699 args = [] if params is None else [params]
-> 2700 cur = self.con.cursor()
    2701 try:
    2702     cur.execute(sql, *args)

ProgrammingError: Cannot operate on a closed database.

```

```
[12]: conn = sqlite3.connect("store.db")
```

```

query = """
SELECT
    c.customer_id,
    c.name,

```

```

        c.city,
        COUNT(o.order_id) AS total_orders,
        ROUND(SUM(o.amount), 2) AS lifetime_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY lifetime_spend DESC
LIMIT 5;
"""
top_customers_df = pd.read_sql_query(query, conn)

conn.close()

print(top_customers_df)

```

	customer_id	name	city	total_orders	lifetime_spend
0	29	Iris Ali	Culver City	7	1222.96
1	30	Jon Brooks	Los Angeles	8	1177.50
2	9	Iris Ali	Inglewood	8	1091.49
3	13	Marco Nguyen	Los Angeles	8	978.99
4	11	Kai Lopez	Los Angeles	5	955.14

```

[13]: import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

orders_df = pd.read_csv("store_orders.csv")
customers_df = pd.read_csv("store_customers.csv")

con = sqlite3.connect("store.db")
orders_df.to_sql("orders", con, if_exists="replace", index=False)
customers_df.to_sql("customers", con, if_exists="replace", index=False)

query = """
SELECT
    c.city,
    SUM(o.amount) AS total_spend
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_spend DESC;
"""

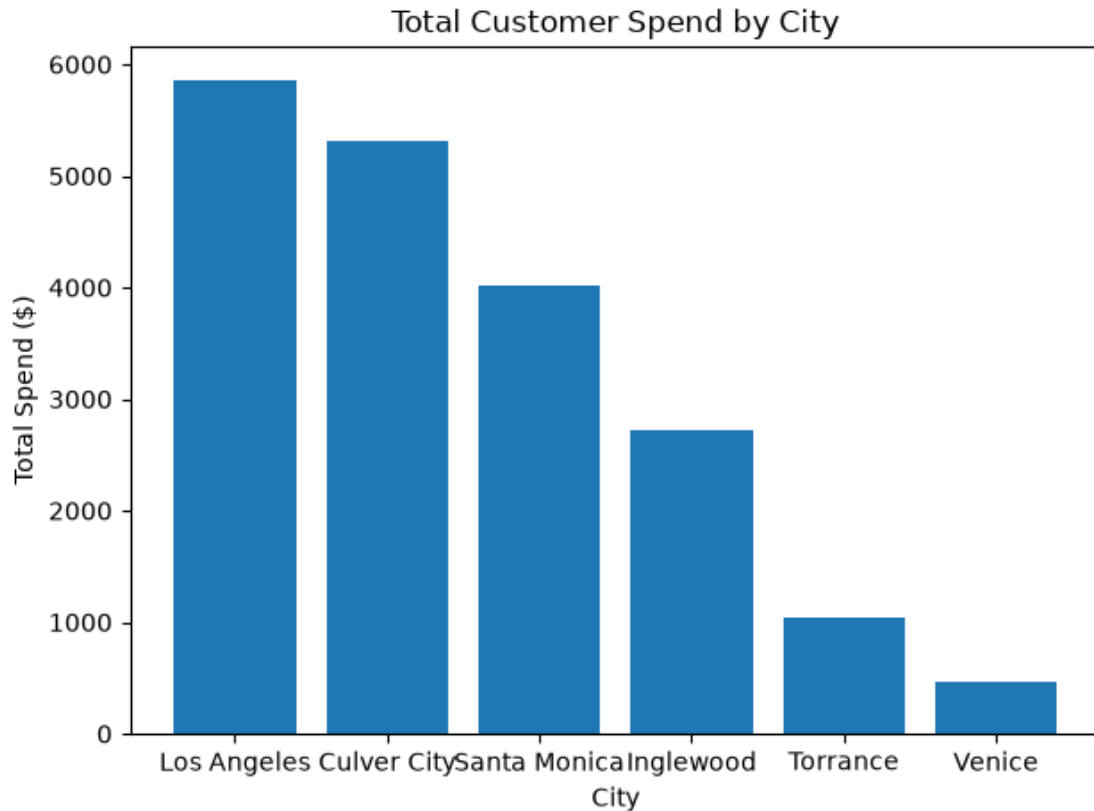
df_city_spend = pd.read_sql(query, con)

plt.figure
bars = plt.bar(df_city_spend['city'], df_city_spend['total_spend'])

```

```
plt.title('Total Customer Spend by City')
plt.xlabel('City')
plt.ylabel('Total Spend ($)')

plt.tight_layout()
plt.savefig('spend_by_city.png', dpi=300)
plt.show()
```



A business question I would ask is, “who are our biggest customers and where are they from?”

```
SELECT c.customer_id, c.name, c.city, COUNT(o.order_id) AS total_orders,
ROUND(SUM(o.amount), 2) AS lifetime_spend FROM customers c JOIN orders o ON
c.customer_id = o.customer_id GROUP BY c.customer_id, c.name, c.city ORDER BY
lifetime_spend DESC LIMIT 5;
```

I would redact to remove columns like Customer Names and Location because these are private information, and they can be identified easily by using these two information.

[]: